

1. Qu'est-ce qu'un middleware en Express.js ?

Un **middleware** est une fonction qui s'exécute **entre la requête et la réponse**.

Rôle des middlewares :

- Analyser le body (`express.json`)
- Authentifier l'utilisateur (JWT)
- Autoriser l'accès (rôles)
- Gérer la sécurité
- Gérer les erreurs

Schéma :

Requête → Middleware(s) → Route → Réponse

2. Initialisation du projet

2.2 Installer les dépendances

```
npm install bcryptjs jsonwebtoken
```

3. Structure recommandée du projet

```
express-auth/  
|  
├─ config/
```

```
|   └─ db.js
|
|─ controllers/
|   └─ authController.js
|
|─ middlewares/
|   ├── authMiddleware.js
|   └─ roleMiddleware.js
|
|─ models/
|   └─ User.js
|
|─ routes/
|   └─ authRoutes.js
|
|─ .env
|─ server.js
```

4. Configuration de l'environnement

Fichier **.env**

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/authdb
JWT_SECRET=supersecretkey
```

5. Connexion à la base de données

config/db.js

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB connectée");
  }
}
```

```
    } catch (error) {  
      console.error(error.message);  
      process.exit(1);  
    }  
  };  
};
```

```
module.exports = connectDB;
```

6. Serveur Express et middlewares globaux

server.js

```
const express = require("express");  
const dotenv = require("dotenv");  
const cors = require("cors");  
const connectDB = require("../config/db");  
  
dotenv.config();  
connectDB();  
  
const app = express();  
  
/* Middlewares globaux */  
app.use(express.json()); // lire le body JSON  
app.use(cors());          // autoriser les requêtes externes  
  
app.use("/api/auth", require("../routes/authRoutes"));  
  
const PORT = process.env.PORT || 5000;  
app.listen(PORT, () =>  
  console.log(`Serveur lancé sur le port ${PORT}`)  
);
```

7. Modèle User avec rôles

models/User.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },

  email: {
    type: String,
    required: true,
    unique: true
  },

  password: {
    type: String,
    required: true
  },

  role: {
    type: String,
    enum: ["user", "admin"],
    default: "user"
  }
});

module.exports = mongoose.model("User", userSchema);
```

8. AuthController (Register & Login)

controllers/authController.js

8.1 Inscription (Register)

```
const User = require("../models/User");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
```

```
exports.register = async (req, res) => {
  const { name, email, password } = req.body;

  try {
    const userExiste = await User.findOne({ email });
    if (userExiste) {
      return res.status(400).json({ message: "Utilisateur déjà
existant" });
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    await User.create({
      name,
      email,
      password: hashedPassword
    });

    res.status(201).json({ message: "Inscription réussie" });

  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

8.2 Connexion (Login)

```
exports.login = async (req, res) => {
  const { email, password } = req.body;

  try {
    const user = await User.findOne({ email });
    if (!user) {
      return res.status(400).json({ message: "Identifiants
invalides" });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
```

```

        return res.status(400).json({ message: "Identifiants
invalides" });
    }

    const token = jwt.sign(
        { id: user._id, role: user.role },
        process.env.JWT_SECRET,
        { expiresIn: "1j" }
    );

    res.json({
        token,
        user: {
            id: user._id,
            name: user.name,
            email: user.email,
            role: user.role
        }
    });

} catch (error) {
    res.status(500).json({ message: error.message });
}
};

```

9. Middleware d'authentification (JWT)

middlewares/authMiddleware.js

```

const jwt = require("jsonwebtoken");

const protect = (req, res, next) => {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith("Bearer ")) {
        return res.status(401).json({ message: "Accès non autorisé" });
    }

    try {

```

```
const token = authHeader.split(" ")[1];
const decoded = jwt.verify(token, process.env.JWT_SECRET);

req.user = {
  id: decoded.id,
  role: decoded.role
};

next();
} catch (error) {
  res.status(401).json({ message: "Token invalide" });
}
};

module.exports = protect;
```

10. Middleware de rôles (Admin / User)

middleware/roleMiddleware.js

```
const authorize = (roles = []) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        message: "Accès refusé"
      });
    }
    next();
  };
};

module.exports = authorize;
```

11. Routes protégées

routes/authRoutes.js

```
const express = require("express");
const router = express.Router();

const { register, login } =
  require("../controllers/authController");
const protect = require("../middleware/authMiddleware");
const authorize = require("../middleware/roleMiddleware");

router.post("/register", register);
router.post("/login", login);
router.post("/ajouterCour", protect , courController.ajouterCour);

router.get("/list", protect, authorize(["parent", "admin"]), (req,
res) => {
  res.json({ message: "Profil utilisateur", user: req.user });
});

router.get(
  "/admin",
  protect,
  authorize(["admin"]),
  (req, res) => {
    res.json({ message: "Espace administrateur" });
  }
);

module.exports = router;
```